

Introduction to Classical Machine Learning

QML

Contents

1	Why Classical ML in a Quantum Course?	3
2	The Machine Learning Landscape	3
2.1	Supervised Learning	3
2.2	Unsupervised Learning	3
2.3	Reinforcement Learning (Brief)	4
3	The Learning Framework	4
3.1	Hypothesis Class and Model	4
3.2	Loss Functions	4
4	Bias-Variance Tradeoff	5
4.1	Decomposition	5
5	Overfitting and Regularization	6
5.1	Overfitting	6
5.2	Regularization Techniques	6
6	Gradient Descent and Optimization	7
6.1	Gradient Descent	7
6.2	Variants of Gradient Descent	7
7	Model Evaluation	8
7.1	Train/Test/Validation Split	8
7.2	Cross-Validation	8
7.3	Evaluation Metrics for Classification	8
7.4	Evaluation Metrics for Regression	9
8	Key Classical ML Models (Preview)	9
8.1	Linear Models	9
8.2	Decision Trees and Ensembles	10
8.3	Support Vector Machines	10
8.4	Neural Networks	10

8.5	Kernel Methods	10
9	Statistical Learning Theory Basics	11
9.1	Generalization	11
9.2	VC Dimension	11
10	Dimensionality Reduction	12
10.1	Principal Component Analysis (PCA)	12
11	Summary	12

1 Why Classical ML in a Quantum Course?

before u can understand quantum machine learning, u need to understand machine learning. QML is not a replacement for classical ML — its an extension. the same concepts (loss functions, optimization, generalization, bias-variance tradeoff) apply directly. if u dont know classical ML well, QML will just be confusing notation on top of confusion.

this lecture covers the classical ML foundations that the rest of the course builds on. everything here has a quantum analogue that well encounter in later lectures.

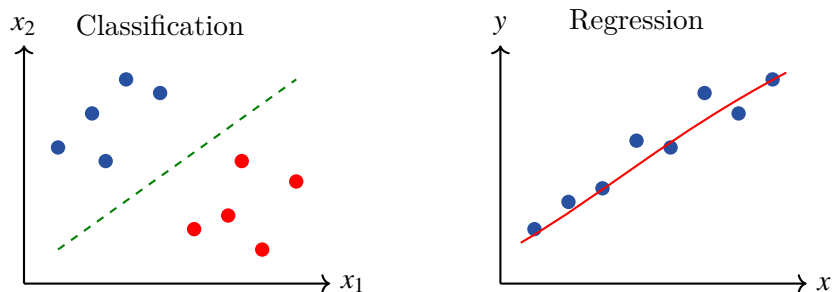
2 The Machine Learning Landscape

2.1 Supervised Learning

Definition 2.1. in supervised learning, u have a dataset of input-output pairs $\{(x_i, y_i)\}_{i=1}^N$ and want to learn a function $f: X \rightarrow Y$ that maps inputs to outputs. the goal is to generalize — predict y for new, unseen x .

two main flavors:

- classification: y is discrete (a label). e.g., is this email spam or not? is this image a cat or dog?
- regression: y is continuous. e.g., predict house prices, stock returns, temperature.



2.2 Unsupervised Learning

Definition 2.2. in unsupervised learning, u only have inputs $\{x_i\}$ — no labels. the goal is to discover structure in the data.

main tasks:

- clustering: group similar data points together (e.g., k-means, DBSCAN)
- dimensionality reduction: find low-dimensional representations (e.g., PCA, t-SNE, autoencoders)
- density estimation: learn the probability distribution of the data (e.g., GMMs, normalizing flows)
- generative modeling: learn to generate new data that looks like the training data (e.g., GANs, VAEs)

Key Point. for QML:

- quantum classifiers (variational quantum classifiers, quantum kernel classifiers) → supervised classification
- quantum generative adversarial networks (QGANs) → generative modeling
- quantum kernel methods → both classification and regression
- quantum autoencoders → dimensionality reduction
- quantum boltzmann machines → density estimation / generative modeling

2.3 Reinforcement Learning (Brief)

Definition 2.3. in reinforcement learning (RL), an agent interacts with an environment, taking actions and receiving rewards. the goal is to learn a policy that maximizes cumulative reward.

RL is less directly connected to the QML methods covered in this course, but there is quantum RL research (using quantum circuits as policy networks, quantum advantage in certain RL settings). we wont go deep into it here.

3 The Learning Framework

3.1 Hypothesis Class and Model

Definition 3.1. a hypothesis class $\mathcal{H}$ is the set of all functions ur model can represent. a model with parameters θ defines a specific function $f_\theta \in \mathcal{H}$.

Example 3.1. • linear models: $\mathcal{H} = \{f(x) = w^T x + b\}$ — all affine functions

- polynomial models: $\mathcal{H} = \{f(x) = \sum_{k=0}^d a_k x^k\}$ — polynomials up to degree d
- neural networks: $\mathcal{H} = \{f_\theta(x)\}$ where θ r the weights and biases — very large (often over-parameterized) hypothesis classes
- quantum models: $\mathcal{H} = \{\text{tr}(O \cdot U(\theta, x) \rho_0 U^\dagger(\theta, x))\}$ — functions computed by parameterized quantum circuits

3.2 Loss Functions

Definition 3.2. a loss function $\mathcal{L}(f_\theta, x, y)$ measures how badly the model's prediction $f_\theta(x)$ disagrees with the true label y . the goal is to minimize the empirical risk (average loss over training data):

$$R_{\text{emp}}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i)$$

common loss functions:

mean squared error (MSE) — for regression:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2$$

cross-entropy loss — for classification:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where $p_i = f_{\theta}(x_i)$ is the predicted probability.

hinge loss — for SVMs:

$$\mathcal{L}_{\text{hinge}} = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i \cdot f_{\theta}(x_i))$$

where $y_i \in \{-1, +1\}$.

Intuition. the choice of loss function encodes ur assumptions:

- MSE assumes gaussian noise and penalizes large errors quadratically
- cross-entropy assumes bernoulli/categorical distributions and is well-calibrated for probabilities
- hinge loss only cares abt the margin (distance from the decision boundary) and ignores correctly classified points far from the boundary

in QML, loss functions r typically expectation values of observables: $\mathcal{L} = \langle O \rangle$ or functions thereof. the connection between classical and quantum loss functions is important for understanding variational algorithms.

4 Bias-Variance Tradeoff

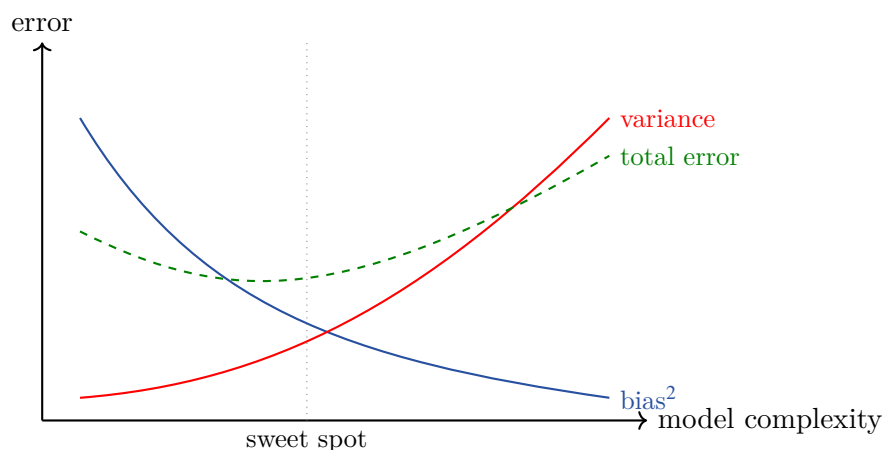
4.1 Decomposition

the expected test error of a model can be decomposed as:

$$\text{expected error} = \text{bias}^2 + \text{variance} + \text{irreducible noise}$$

Definition 4.1. • bias: error from wrong assumptions in the model. high bias = underfitting — the model is too simple to capture the true pattern.

- variance: error from sensitivity to training data fluctuations. high variance = overfitting — the model memorizes noise in the training data.
- irreducible noise: inherent randomness in the data that no model can capture.



Key Point. for QML, the bias-variance tradeoff maps directly:

- circuit too shallow (few parameters): high bias, underfitting. the quantum model cant express the target function.
- circuit too deep (many parameters): high variance, overfitting. the model fits noise. also noise from the hardware adds variance.
- additional complication: quantum noise adds a third source of error beyond classical bias and variance. this is unique to QML on NISQ hardware.

5 Overfitting and Regularization

5.1 Overfitting

Definition 5.1. overfitting occurs when the model performs well on training data but poorly on test data. the model has learned the noise in the training set rather than the underlying pattern.

Example 5.1. fitting a degree-20 polynomial to 10 data points: the polynomial passes through all training points perfectly (training error = 0) but oscillates wildly between points, giving terrible predictions on new data.

5.2 Regularization Techniques

L2 regularization (Ridge): add a penalty on the magnitude of the parameters:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \|\boldsymbol{\theta}\|_2^2$$

discourages large weights, leading to smoother functions.

L1 regularization (Lasso): add a penalty on the absolute values:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \|\boldsymbol{\theta}\|_1$$

encourages sparse solutions (many weights become exactly 0).

dropout (for neural networks): randomly set a fraction of neurons to 0 during training. forces the network to learn redundant representations.

early stopping: stop training before the model overfits, based on validation set performance.

Intuition. regularization in QML is less well-studied than in classical ML but equally important:

- quantum circuit noise itself acts as a form of regularization (injects randomness, smooths the loss landscape)
- restricting circuit depth is a form of regularization (limits model complexity)
- data augmentation and overparameterization interact with quantum models in non-obvious ways
- the paper by caro et al. (2022) shows that quantum models can generalize from few training data, partly bcs the quantum hypothesis class has favorable structure

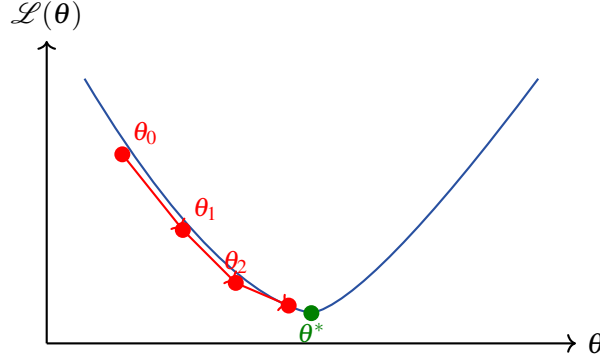
6 Gradient Descent and Optimization

6.1 Gradient Descent

Definition 6.1. gradient descent (GD) iteratively updates parameters to minimize the loss:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

where η is the learning rate.



6.2 Variants of Gradient Descent

stochastic gradient descent (SGD): instead of computing the gradient over the entire dataset, use a random mini-batch:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\text{batch}}(\theta_t)$$

faster per iteration, introduces noise that can help escape local minima.

SGD with momentum: add a “velocity” term:

$$v_{t+1} = \beta v_t + \nabla_{\theta} \mathcal{L}(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t - \eta v_{t+1} \quad (2)$$

smooths out oscillations and accelerates convergence.

Adam (Adaptive Moment Estimation): adapts the learning rate for each parameter based on first and second moments of the gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (5)$$

where $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$, $\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$.

Key Point. optimization in QML:

- quantum circuit parameters θ r optimized classically using gradient descent or gradient-free methods
- gradients can be computed using the parameter shift rule: $\frac{\partial \langle O \rangle}{\partial \theta_i} = \frac{\langle O \rangle_{\theta_i + \pi/2} - \langle O \rangle_{\theta_i - \pi/2}}{2}$
- each gradient evaluation requires running the quantum circuit twice per parameter

- total cost for one optimization step: $2p$ circuit evaluations (where p is the number of parameters)
- noisy gradients from shot noise and hardware noise make optimization harder
- adam and other adaptive methods r commonly used in variational quantum algorithms

7 Model Evaluation

7.1 Train/Test/Validation Split

Definition 7.1. split ur data into three sets:

- training set ($\sim 60\text{--}80\%$): used to train the model (update parameters)
- validation set ($\sim 10\text{--}20\%$): used to tune hyperparameters (learning rate, circuit depth, etc.) and detect overfitting
- test set ($\sim 10\text{--}20\%$): used only once at the end to estimate generalization performance

7.2 Cross-Validation

Definition 7.2. k -fold cross-validation: split data into k folds. train on $k - 1$ folds, validate on the remaining fold. repeat k times (each fold serves as validation once). average the k validation scores.

cross-validation gives a more reliable estimate of generalization performance than a single train/test split, especially with small datasets (common in QML research).

7.3 Evaluation Metrics for Classification

accuracy: fraction of correct predictions.

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

can be misleading for imbalanced datasets.

confusion matrix: for binary classification:

$$\begin{pmatrix} \text{TP} & \text{FP} \\ \text{FN} & \text{TN} \end{pmatrix}$$

precision: of all predicted positives, how many r actually positive?

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

recall (sensitivity): of all actual positives, how many did we catch?

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1 score: harmonic mean of precision and recall.

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

ROC-AUC: area under the receiver operating characteristic curve. plots true positive rate vs false positive rate at various classification thresholds. $AUC = 1$ is perfect, $AUC = 0.5$ is random guessing.

Example 7.1. suppose a quantum classifier produces:

- $TP = 45, FP = 5, FN = 10, TN = 40$

$$\text{accuracy} = (45 + 40)/100 = 85\%$$

$$\text{precision} = 45/(45 + 5) = 90\%$$

$$\text{recall} = 45/(45 + 10) = 81.8\%$$

$$F1 = 2 \times 0.9 \times 0.818 / (0.9 + 0.818) = 85.7\%$$

7.4 Evaluation Metrics for Regression

MSE: mean squared error (already defined).

MAE: mean absolute error $= \frac{1}{N} \sum |f(x_i) - y_i|$.

R^2 score (coefficient of determination):

$$R^2 = 1 - \frac{\sum (y_i - f(x_i))^2}{\sum (y_i - \bar{y})^2}$$

$R^2 = 1$ is perfect, $R^2 = 0$ means the model predicts the mean, $R^2 < 0$ means worse than predicting the mean.

8 Key Classical ML Models (Preview)

we'll go deep into SVMs and kernel methods in lecture 8 (bcs they connect directly to quantum kernels). here's a quick overview of the main model families:

8.1 Linear Models

$$f(x) = w^T x + b$$

- linear regression: minimize MSE
- logistic regression: $\sigma(w^T x + b)$ for binary classification, minimize cross-entropy
- perceptron: simplest neural network unit

simple, interpretable, fast. but limited to linearly separable problems.

8.2 Decision Trees and Ensembles

- decision trees: split the feature space into regions using if/then rules
- random forests: ensemble of trees (bootstrap aggregating)
- gradient boosting (XGBoost, LightGBM): sequentially build trees that correct previous trees' errors

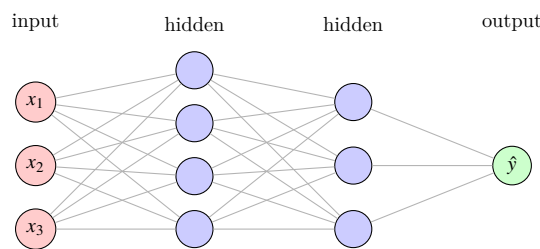
extremely effective for tabular data. often the best choice for structured data even today.

8.3 Support Vector Machines

separate data with a maximum-margin hyperplane. use the kernel trick to handle non-linear decision boundaries. deep connection to quantum kernels — we'll cover this extensively in lecture 8.

8.4 Neural Networks

a simple feedforward neural network (MLP):



each connection has a weight, each node applies a nonlinear activation function σ (ReLU, sigmoid, tanh). the output is:

$$\hat{y} = \sigma(W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 x + b_1) + b_2) + b_3)$$

- MLP: stacked linear layers with nonlinear activations
- CNN: convolutional layers for spatial data (images)
- RNN/LSTM/Transformer: sequential/attention-based architectures for sequence data

the dominant paradigm in modern ML. quantum neural networks (PQCs) are inspired by classical NNs but have fundamentally different properties (unitary transformations, entanglement, exponential state space).

8.5 Kernel Methods

map data to a high-dimensional feature space and use linear methods there. the kernel trick avoids explicitly computing the feature map. this is the direct classical ancestor of quantum kernel methods.

Key Point. the two main paradigms of QML mirror classical ML:

1. quantum kernel methods: the quantum computer computes a kernel function $K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ that encodes the data in a quantum feature space. classification/regression is done classically using this kernel.
2. variational quantum models: analogous to neural networks. parameterized quantum circuits with trainable parameters, optimized end-to-end. the quantum equivalent of “learn the feature map and the classifier jointly.”

schuld (2021) showed that both paradigms are mathematically equivalent (all variational models can be seen as kernel methods), but they differ in practice (training dynamics, computational cost, generalization).

9 Statistical Learning Theory Basics

9.1 Generalization

the central question of ML: if the model does well on training data, will it do well on new, unseen data?

Definition 9.1. the generalization error (true risk) is:

$$R(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\mathcal{L}(f_\theta(x), y)]$$

where $\mathcal{D}$ is the true data distribution. we can't compute this directly (we don't know $\mathcal{D}$), so we estimate it with the test set.

generalization gap: $R(\theta) - R_{\text{emp}}(\theta)$ — the difference between true risk and empirical (training) risk. small gap = good generalization.

9.2 VC Dimension

Definition 9.2. the VC dimension of a hypothesis class $\mathcal{H}$ is the largest set of points that $\mathcal{H}$ can shatter (classify correctly for all possible labelings).

Example 9.1. • linear classifiers in $\mathbb{R}^2$: VC dimension = 3 (can shatter any 3 non-collinear points, can't shatter 4 points)

- linear classifiers in $\mathbb{R}^d$: VC dimension = $d + 1$
- single-layer neural network with p parameters: VC dimension $\sim O(p)$

the VC dimension bounds generalization:

$$R(\theta) \leq R_{\text{emp}}(\theta) + O\left(\sqrt{\frac{d_{\text{VC}} \log(N/d_{\text{VC}})}{N}}\right)$$

so: if $N \gg d_{\text{VC}}$, the generalization gap is small.

Key Point. for QML, caro et al. (2022) proved generalization bounds showing that quantum models with t trainable gates can learn from $O(t)$ training examples. this is favorable — it means quantum models don't necessarily need huge datasets to generalize, which is important bcs preparing large quantum datasets can be expensive.

10 Dimensionality Reduction

10.1 Principal Component Analysis (PCA)

Definition 10.1. PCA finds the directions of maximum variance in the data. project the data onto the top k principal components (eigenvectors of the covariance matrix with the largest eigenvalues).

$$\text{covariance matrix: } \Sigma = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T$$

eigendecompose $\Sigma = V\Lambda V^T$. the first k columns of V define the projection.

Intuition. PCA has a quantum analogue: quantum PCA uses QPE to extract the eigenvalues of the quantum state's density matrix. this can be exponentially faster than classical PCA for certain input models (the HHL paper connects here). but the input assumptions r strong and debated.

11 Summary

Key Point. classical ML concepts that carry over directly to QML:

1. supervised learning: learn $f : X \rightarrow Y$ from labeled data. quantum version: variational quantum classifiers, quantum kernel classifiers.
2. loss functions: MSE, cross-entropy, hinge. quantum version: expectation values of observables.
3. gradient descent: iterative parameter updates. quantum version: parameter shift rule for quantum gradients.
4. bias-variance tradeoff: circuit depth controls model complexity. too shallow = underfitting, too deep = overfitting + noise.
5. regularization: L2 penalty, dropout, early stopping. quantum version: circuit depth restriction, noise as regularizer.
6. evaluation metrics: accuracy, precision, recall, F1, AUC — same metrics used for quantum classifiers.
7. generalization theory: VC dimension, generalization bounds. quantum version: quantum VC dimension, effective dimension.
8. kernel methods: the bridge between classical and quantum ML. quantum kernel methods r a direct extension of classical kernel methods.

understanding these classical concepts is essential. QML doesnt replace them — it extends them.

next: SVMs, kernel methods in depth, and the neural network perspective. this is where the connection to quantum ML becomes explicit.